

SENTINELMRO

Decentralized Edge-AI & Cryptographic Auditing Ecosystem for Predictive Maintenance

Project Target: Tata Technologies InnoVent-27 Competition

Category: 3.2.3.3 (Decentralized Prognostics & Tamper-Evident Ledger)

Technology Stack: Next.js 15, FastAPI, PyTorch, ONNX INT8 Quantization, SQLite, Ed25519 Cryptography

Version: 1.0.0 (Production Prototype)

Date: July 2026

CONFIDENTIALITY NOTICE: The information contained in this technical document is proprietary to the SentinelMRO Engineering Team and prepared strictly for evaluation under the Tata Technologies InnoVent-27 guidelines.

1. Executive Summary

Aviation Maintenance, Repair, and Overhaul (MRO) activities rely heavily on accurate predictive diagnostics to minimize unplanned aircraft groundings (AOG) while maintaining impeccable safety records. Historically, these systems use centralized cloud storage. However, centralized infrastructures suffer from critical bottlenecks, including high data transfer bandwidths, airline operational privacy concerns, and database susceptibility to unauthorized modifications or tampering.

SentinelMRO resolves these constraints by implementing a decentralized paradigm. The system runs localized, low-latency **INT8-quantized Temporal Convolutional Networks (TCN)** directly on edge nodes (simulated MRO stations). To continuously train and improve these models without leaking proprietary operational telemetry, SentinelMRO coordinates cross-station **Federated Learning** using the **FedProx** optimization algorithm, supplemented by **Local Differential Privacy (LDP)**. Furthermore, flight logs, sensor data snapshots, and maintenance history are cryptographically signed using **Ed25519 keys** and recorded in an append-only **Merkle Mountain Range (MMR) ledger**, providing tamper-evidence and rapid mathematical verification of data integrity.

2. System Architecture & Topology

The SentinelMRO ecosystem comprises three key architectural layers:

- **Edge Nodes (FastAPI + ONNX Runtime):** Serve real-time inference on 14 selected sensor channels in under 3ms. Features are normalized, sliced, and loaded into an INT8-quantized TCN model.
- **Aggregation Coordinator (FastAPI Gateway):** Directs federated learning training rounds, implements mathematical aggregation (FedProx averaging), and exposes metrics.
- **SQLite Merkle Mountain Range Ledger:** Houses signed records. The MMR database is maintained in a local SQLite file (ledger.db) for O(log n) audits.

3. Edge-AI & Quantization Pipeline (Phase 1)

SentinelMRO uses the NASA C-MAPSS dataset (FD001 engine trajectories) as its ML benchmark. Before training, low-variance sensor readings are stripped out to compress input dimension.

Sensor Type	Action	Sensor Channels	Variance / Details
Low-Variance Constants	Dropped	s1, s5, s6, s10, s16, s18, s19	Flatlines / No operational variance
High-Variance Dynamics	Retained	s2, s3, s4, s7, s8, s9, s11, s12, s13, s14, s15, s17, s20, s21	Active sensors mapped to sliding windows

Model Compiling and Quantization: A causal TCN (Temporal Convolutional Network) is trained on PyTorch, implementing exponential dilations of 1, 2, 4, and 8. Weight normalization is used during optimization. For edge deployment, we remove weight normalization hooks (collapsing them into static parameters) and compile the network into an ONNX graph. Post-training dynamic INT8 quantization is applied, achieving a **4x file size reduction** and sub-3ms CPU execution.

4. Federated Optimization (Phase 2)

Under non-IID conditions where data varies widely between airlines and MRO hubs, standard Federated Averaging (FedAvg) leads to poor model convergence. SentinelMRO implements the **FedProx** optimization routine to handle this heterogeneity. The local optimization loss is regularized using a proximal constraint:

FedProx Loss Formula:

$$\text{Loss_prox}(w) = \text{Loss_local}(w) + (\mu / 2) * || w - w^t ||^2$$

Where w represents the local parameters, w^t is the global model weights at round t , and μ is set to 0.01. This mathematical penalty stabilizes weight adjustments, preventing local weights from drifting too far from the global average.

Local Differential Privacy (LDP) is applied by adding calibrated Gaussian noise directly to the model weights before uploading: $w_{\text{priv}} = w_{\text{local}} + N(0, 0.001^2)$, ensuring that raw telemetry data cannot be reconstructed via model inversion attacks.

5. SQLite Cryptographic MMR Ledger (Phase 3)

Every event sent to the central coordinator must be cryptographically signed by the submitting station's private Ed25519 key. The coordinator validates this signature against the station's known public key to securely verify identity.

The maintenance transactions are logged into an append-only SQLite database using a **Merkle Mountain Range (MMR)** structure. Unlike standard linear chains, MMR enables $O(\log n)$ inclusion proofs and checks. If any database column is altered maliciously, the root hash is broken and integrity validation immediately triggers an alarm.

MMR Ledger Database Table Schema (mro_ledger):

Column Name	Data Type	Constraints	Description
leaf_index	INTEGER	PRIMARY KEY	Auto-incrementing leaf position (1-indexed)
timestamp	TEXT	NOT NULL	ISO-8601 string of log entry time
component_id	TEXT	NOT NULL	ID of target component (e.g., ENG-001)
action_taken	TEXT	NOT NULL	Maintenance detail (e.g., Blade Replacement)
technician_id	TEXT	NOT NULL	ID of MRO technician
health_snapshot	REAL	NOT NULL	TCN-inferred continuous health score (1.0 to 0.0)
node_hash	TEXT	NOT NULL	SHA-256 hash of the row payload (excluding index)

6. Worked Trace Example: Logging & Tampering

To illustrate the end-to-end operation, let us walk through a complete logged event and a subsequent security violation check.

Step 1: Technician replacement entry

A technician replaces a compressor blade on ENG-002. The edge node constructs the message payload: '2026-07-03T13:42:53.000Z|ENG-002|Blade Replacement|TECH-502|0.85'

Step 2: Ed25519 Signing

The edge node uses STATION_001's private key to sign the message, yielding signature hex '8bf982cd6b4ef8919e...'. This is submitted to the API gateway in the HTTP headers.

Step 3: Verification & Append

The API gateway verifies the signature using STATION_001's public key. Upon confirmation, the payload is written to mro_ledger. The MMR computes the leaf hash $\text{sha256}(\text{payload}) = \text{'5f8a0dc4f67c...'}$ and merges peaks, producing a new cryptographic root hash.

Step 4: Tampering & Auditing

If a malicious actor directly updates SQLite to change `health_snapshot = 0.99` on this row, running the `/verify` routine will re-calculate the leaf hash from the actual row columns. The recomputed hash (starting with '7a11c8...') will mismatch the stored ``node_hash`` (starting with '5f8a0dc4f67c...'), and the verification fails. The dashboard UI immediately flashes red, notifying MRO managers of data corruption.

7. Project Completion Checklist

- ✓ **Edge Inference:** Quantized TCN model executes locally in < 3ms.
- ✓ **Federated Optimizer:** FedProx updates stable weights across non-IID partitions.
- ✓ **Cryptographic MMR Ledger:** Append, verify, and tamper routes fully functional in SQLite.
- ✓ **Command Console:** Responsive Next.js 15 dashboard showing real-time streaming, aggregation charts, and ledger alerts.